

The BGP Looking Glass

A passive route collector in the SRv6 core — every VPN prefix, its attributes, and how they changed



of every change, kept

Alpine + FRR on 512 MiB · iBGP to both route reflectors · a direct read of all twelve routers
SQLite history · Flask API and page · Prometheus metrics · <http://192.168.50.231:8092>

What it holds

Read from the collector itself while these slides were made — nothing on this page is typed in.

76 / 152

prefixes, paths

the core's VPNv4 + VPNv6 table, not
one router's view

2 / 2

reflector sessions

one link to p1, one to p3 — 60
VPNv4 and 16 VPNv6 accepted on
each, 0 sent

13

routers read directly

their own RIB, VPN and per-VRF
tables, over their API and over SSH

25,235

recorded changes

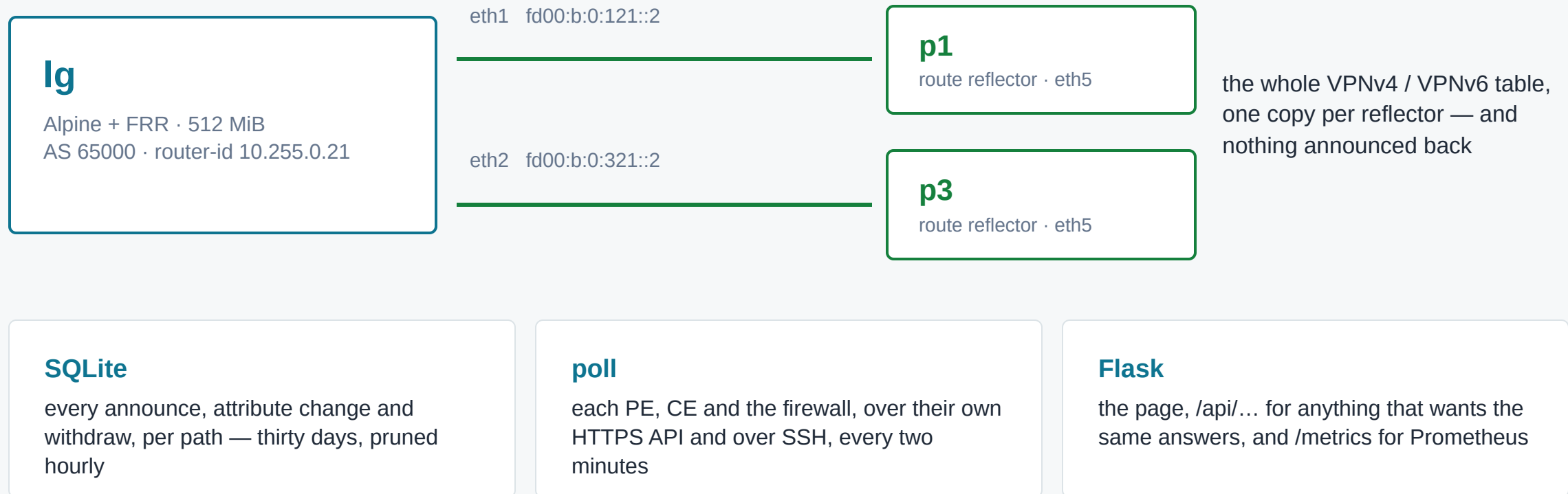
announces, attribute changes and
withdraws, reconciled per path

A looking glass that answers three questions, not one.

What does the core carry, with the attributes the originating PE set? What does each router actually have, after import and after selection? And what did either of those look like at some moment in the past? The first needs a BGP session, the second needs the routers themselves, and the third needs a memory — so it has all three, and says which one every answer came from.

How it is wired

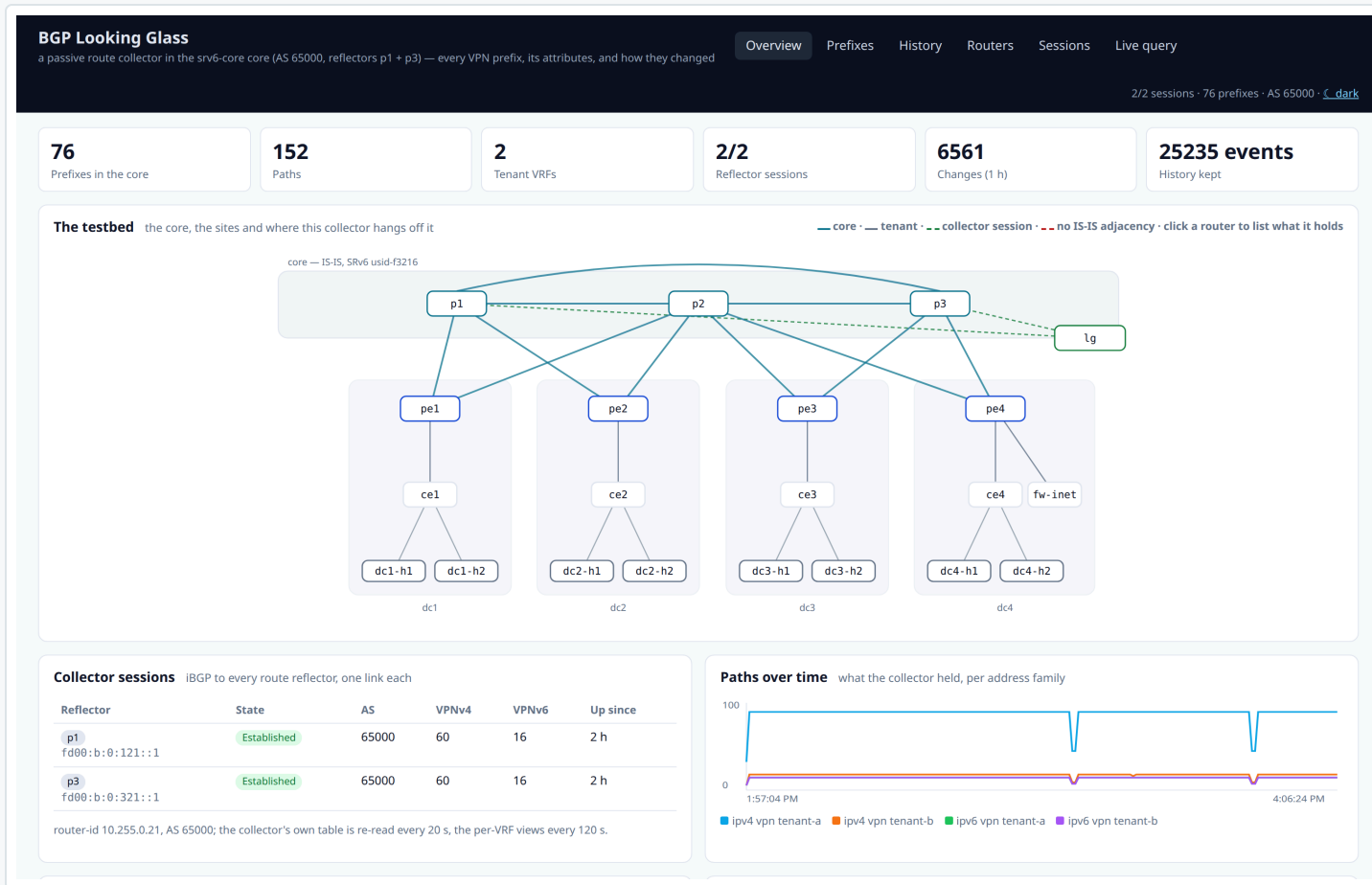
It joins the core as a reflector client on its own two links, and reads every other router besides.



The reflectors' side of both links is rendered by tools/render.py like every other interface, and the VM is modelled in Nautobot like every other device — nautobot render --check compares its frr.conf and its lgd.json too.

The page you land on

What the collector holds, the sessions it holds it over, the testbed, the per-family count over time, what each view holds, and the newest changes.



Read the top row first

Prefixes and paths are the core's table; tenant VRFs is how many the collector has ever seen.

Reflector sessions is the one number that invalidates everything else if it is not 2/2.

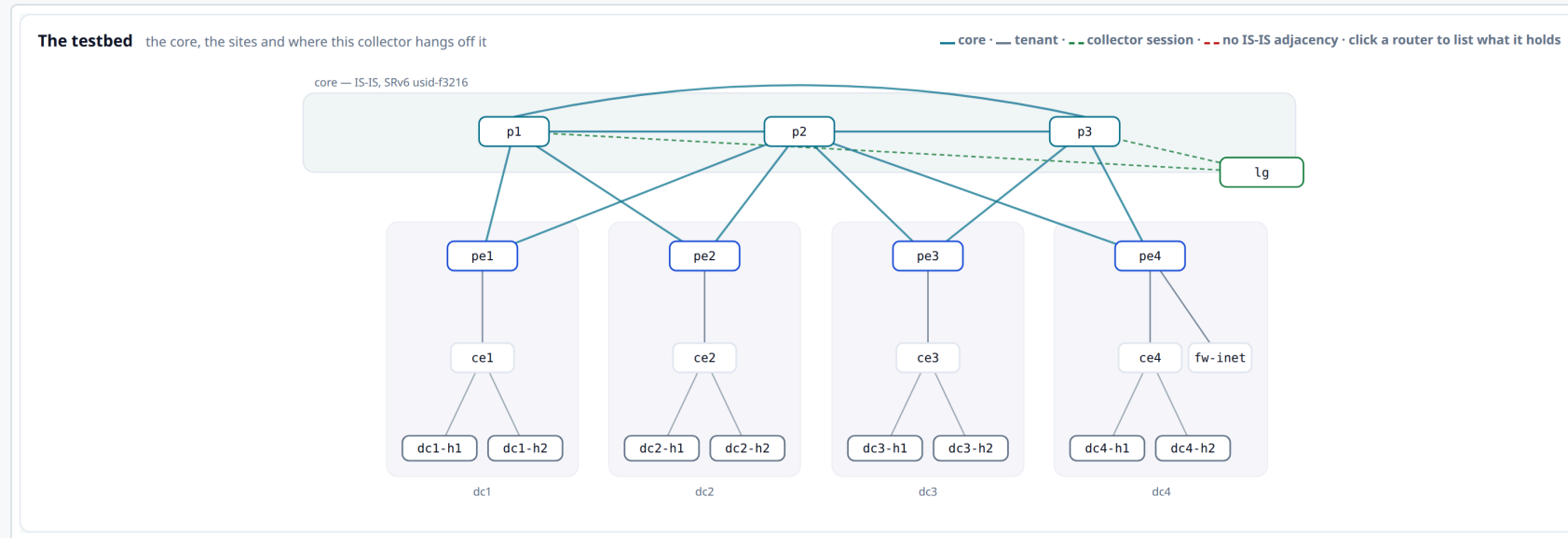
Changes in the last hour is the churn: a quiet core is a flat line, and a noisy one is visible here before anyone reports it.

History kept is the size of the memory behind every question on the following slides.

One page: the answer to “is it healthy and what does it have?” before any question is asked

Where it sits in the core

The collector hangs off both route reflectors on its own point-to-point links, and touches nothing else.



Drawn from the model: the core band with the three P routers, a lane per data centre, and the two green sessions on the right

It is a client, not a peer

p1 and p3 reflect to it as they reflect to a PE — the whole table, with the originating PE's own attributes.

It announces nothing

An outbound deny route-map says so, and both sessions report 0 sent: the core cannot be affected by it.

A red link is a real fault

Core links go red where no router reports an IS-IS adjacency — coloured from what the routers say.

Why a session, and not screen-scraping

The collector receives the same update the reflector sent — route distinguisher, route targets, SRv6 SID and label, originator-id, cluster list, AS path, local preference.

BGP Looking Glass
a passive route collector in the srv6-core core (AS 65000, reflectors p1 + p3) — every VPN prefix, its attributes, and how they changed

OverviewPrefixesHistoryRoutersSessionsLive query

2/2 sessions · 76 prefixes · AS 65000 · [dark](#)

BGP sessions the collector is a reflector client: it receives everything and announces nothing

Peer	Name	State	AS	Router-id	Accepted	Messages rx/tx	Established
fd00:b:0:121::1	p1	Established	65000	10.255.0.11	ipv4Vpn: 60 ipv6Vpn: 16	1142 / 783	2 h ago (1×)
fd00:b:0:321::1	p3	Established	65000	10.255.0.13	ipv4Vpn: 60 ipv6Vpn: 16	1079 / 783	2 h ago (1×)

Collections the collector's own table is read locally; the per-VRF views are polled over SSH

View	Last	Result	Paths	Took	Error
ce1	34 s ago	ok	190	3.6 s	
ce2	34 s ago	ok	190	3.8 s	
ce3	34 s ago	ok	190	3.9 s	
ce4	34 s ago	ok	190	3.8 s	
core (local)	18 s ago	ok	152	0.1 s	
fw-inet	34 s ago	ok	195	3.5 s	
p1	36 s ago	ok	132	2.2 s	
p2	37 s ago	ok	58	1.2 s	
p3	36 s ago	ok	132	1.7 s	
pe1	34 s ago	ok	416	3.9 s	
pe2	34 s ago	ok	416	3.9 s	
pe3	34 s ago	ok	416	3.8 s	
pe4	34 s ago	ok	410	3.8 s	

The subtle part

The session negotiates capability extended-nexthop. Without it FRR rewrites the IPv6 next hop of every VPNv4 route to its own address — and every prefix in the core would look as though the reflector had originated it.

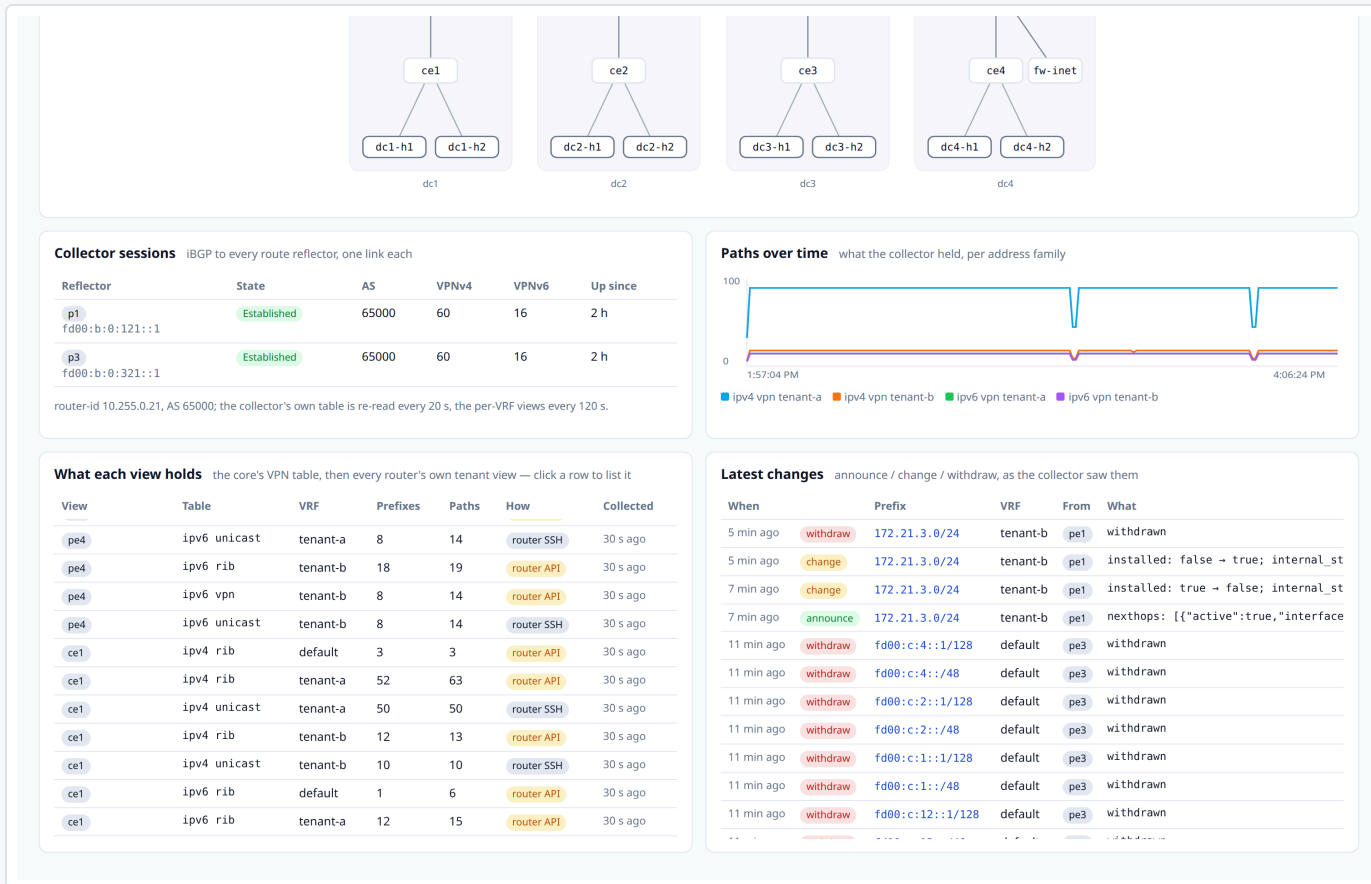
With it, the originating PE's loopback is still the next hop, which is what makes the path on a later slide reconstructable at all.

Screen-scraping a `show` command gives you the text a router chose to print. A session gives you what it sent.

The sessions, and the health of every collection beneath them

And it reads every router directly, too

The core's table says what travels between PEs. What a tenant actually has is the VRF table after import; what the box will do with a packet is in its RIB.



Three transports, one page

rr-session — the whole VPNv4 / VPNv6 table, as the reflectors handed it over.

router-api — each router's own HTTPS API: its RIB per VRF and family, its own VPN table, its IS-IS adjacencies.

router-ssh — the per-VRF BGP tables, where the API has no JSON form for the question and the text form loses the attributes.

Every row, everywhere in the product, carries which one it came from.

What each view holds, per router, VRF and family — with the transport it came over and when it was collected

Its account of its own work

The Routers page: identity, the transports each box was read through, when, how much it holds, and the IS-IS adjacencies it reported.

BGP Looking Glass

a passive route collector in the srv6-core core (AS 65000, reflectors p1 + p3) — every VPN prefix, its attributes, and how they changed

Overview

Prefixes

History

Routers

Sessions

Live query

2/2 sessions · 76 prefixes · AS 65000 · [dark](#)

The routers

every box the looking glass reads, and how it reads each part

router API

 the router's HTTPS API (vYOS 'service https api', JSON) ·

router SSH

 vttysh over SSH, for the views the API cannot express ·

session

 the collector's own iBGP session to the route reflectors

Router	Role	Identity	Read through	Last	RIB	BGP (per VRF)	VPN table	IS-IS adjacencies
ce1 10.3.0.31	ce dc1	rid 172.20.1.1 · AS 65001	router API router SSH	77 s ago 3.6 s	114	76	0	—
ce2 10.3.0.32	ce dc2	rid 172.20.2.1 · AS 65002	router API router SSH	77 s ago 3.8 s	114	76	0	—
ce3 10.3.0.33	ce dc3	rid 172.20.3.1 · AS 65003	router API router SSH	76 s ago 3.9 s	114	76	0	—
ce4 10.3.0.34	ce dc4	rid 172.20.4.1 · AS 65004	router API router SSH	77 s ago 3.8 s	114	76	0	—
fw-inet 10.3.0.61	fw dc4	rid 10.3.0.61 · AS 65010	router API router SSH	77 s ago 3.5 s	135	60	0	—
p1 10.3.0.21	p core	rid 10.255.0.11 · AS 65000 · fd00:a::11 · fd00:c::11::/48	router API	78 s ago 2.2 s	56	0	76	p2 p3 pe1 pe2
p2 10.3.0.22	p core	fd00:a::12 · fd00:c::12::/48	router API	79 s ago 1.2 s	58	0	0	p1 p3 pe1 pe2 pe3 pe4
ce5 10.3.0.35	ce dc5	rid 172.20.5.1 · AS 65005	router API router SSH	77 s ago 3.7 s	114	76	0	—

What each router holds

by address family and VRF — click to list it

Router	Table	VRF	Prefixes	Paths	How
ce1	ipv4 rib	default	3	3	router API
ce1	ipv4 rib	tenant-a	52	63	router API
ce1	ipv4 rib	tenant-b	12	13	router API
ce1	ipv6 rib	default	1	6	router API
ce1	ipv6 rib	tenant-a	12	15	router API
ce1	ipv6 rib	tenant-b	12	14	router API
ce1	ipv4 unicast	tenant-a	50	50	router SSH
ce1	ipv4 unicast	tenant-b	10	10	router SSH
ce1	ipv6 unicast	tenant-a	8	8	router SSH

Collection is a first-class thing

A collection that failed is visible here rather than showing up as a prefix that quietly went missing.

Each router's HTTPS API is rendered into its configuration like the rest of it — key srv6core-lab-looking-glass, reachable only from the collector and the lab host.

So the same interface is open to anything else that wants it; the looking glass is one consumer, not a privileged one.

Thirteen routers, each with its own API key, read on a two-minute cycle

The prefix table

One row per path, with its next hop, AS path, local preference, SRv6 SID and route targets — and, on the right, the view it came from and how it was read.

BGP Looking Glass

a passive route collector in the srv6-core core (AS 65000, reflectors p1 + p3) — every VPN prefix, its attributes, and how they changed

Overview

Prefixed

History

Routers

Sessions

Live query

2/2 sessions · 76 prefixes · AS 65000 · [dark](#)

Prefixed

3087 paths match

View

Address family

VRF

Route distinguisher

How it was read

Origin AS

Search (prefix, next hop, SID, RT, AS path)

every view

every family

every VRF

every RD

however it was

65001

172.20. / fd00:c:: / RT:65000:100

Search

Reset

The collector's view is the core's VPN table: one row per path a reflector handed over, with the attributes the PE originated.

	Prefix	VRF	RD	Next hop	From	AS path	Origin	Local pref	MED	SID	Route targets	Age	View	How
path	0.0.0.0/0 best	default	—	192.168.122.1 fw-inet	local	i	—	—	—	—	—	18 h	fw-inet	router API
path	0.0.0.0/0 best	tenant-a	—	172.16.1.1 ce1	local	i	—	—	—	—	—	18 h	ce1	router API
path	0.0.0.0/0 best	tenant-a	—	172.16.1.1	172.16.1.1	65000 65010	65010	—	0	—	RT:65000:100	26 h	ce1	router SSH
path	0.0.0.0/0 best	tenant-a	—	172.16.2.1 ce2	local	i	—	—	—	—	—	18 h	ce2	router API
path	0.0.0.0/0 best	tenant-a	—	172.16.2.1	172.16.2.1	65000 65010	65010	—	0	—	RT:65000:100	26 h	ce2	router SSH
path	0.0.0.0/0 best	tenant-a	—	172.16.3.1 ce3	local	i	—	—	—	—	—	18 h	ce3	router API
path	0.0.0.0/0 best	tenant-a	—	172.16.3.1	172.16.3.1	65000 65010	65010	—	0	—	RT:65000:100	26 h	ce3	router SSH
path	0.0.0.0/0 best	tenant-a	—	172.16.4.1 ce4	local	i	—	—	—	—	—	18 h	ce4	router API
path	0.0.0.0/0 best	tenant-a	—	172.16.4.1	172.16.4.1	65000 65010	65010	—	—	—	—	26 h	ce4	router SSH
path	0.0.0.0/0 best	tenant-a	65000:104	fd00:a::4 pe4	p1	65010	65010	100	0	fd00:c::4::	RT:65000:100	26 h	core	session
path	0.0.0.0/0 best	tenant-a	65000:104	fd00:a::4 pe4	p1	65010	65010	100	0	fd00:c::4::	RT:65000:100	26 h	core	session

Load more

showing 200 of 3087

The classic questions

“Who originates this?” — origin AS and originating node are columns, not something to derive from an AS path.

“Is it in the tenant's VRF, or only in the core?” — the view filter answers it directly.

“What SID does it carry?” — the egress PE's End.DT46 is on the row, and the free-text box searches it.

Every row has a path button, which is the next slide.

Filters for view, family, VRF, route distinguisher, origin AS and transport, plus free text over prefix, next hop, SID, route target and AS path

Two answers to one question

The same tenant, asked of the reflectors' table and then of the routers' own tables — and the difference is the point.

Prefixes116 paths match

View

Address family

VRF

Route distinguisher

How it was read

Origin AS

Search (prefix, next hop, SID, RT, AS path)

Search

Reset

every view

every family

tenant-a

every RD

session

65001

172.20. / fd00:c:1:: / RT:65000:100

How it was read: session — tenant-a as the reflectors handed it over

Prefixes1166 paths match

View

Address family

VRF

Route distinguisher

How it was read

Origin AS

Search (prefix, next hop, SID, RT, AS path)

Search

Reset

every view

every family

tenant-a

every RD

router API

65001

172.20. / fd00:c:1:: / RT:65000:100

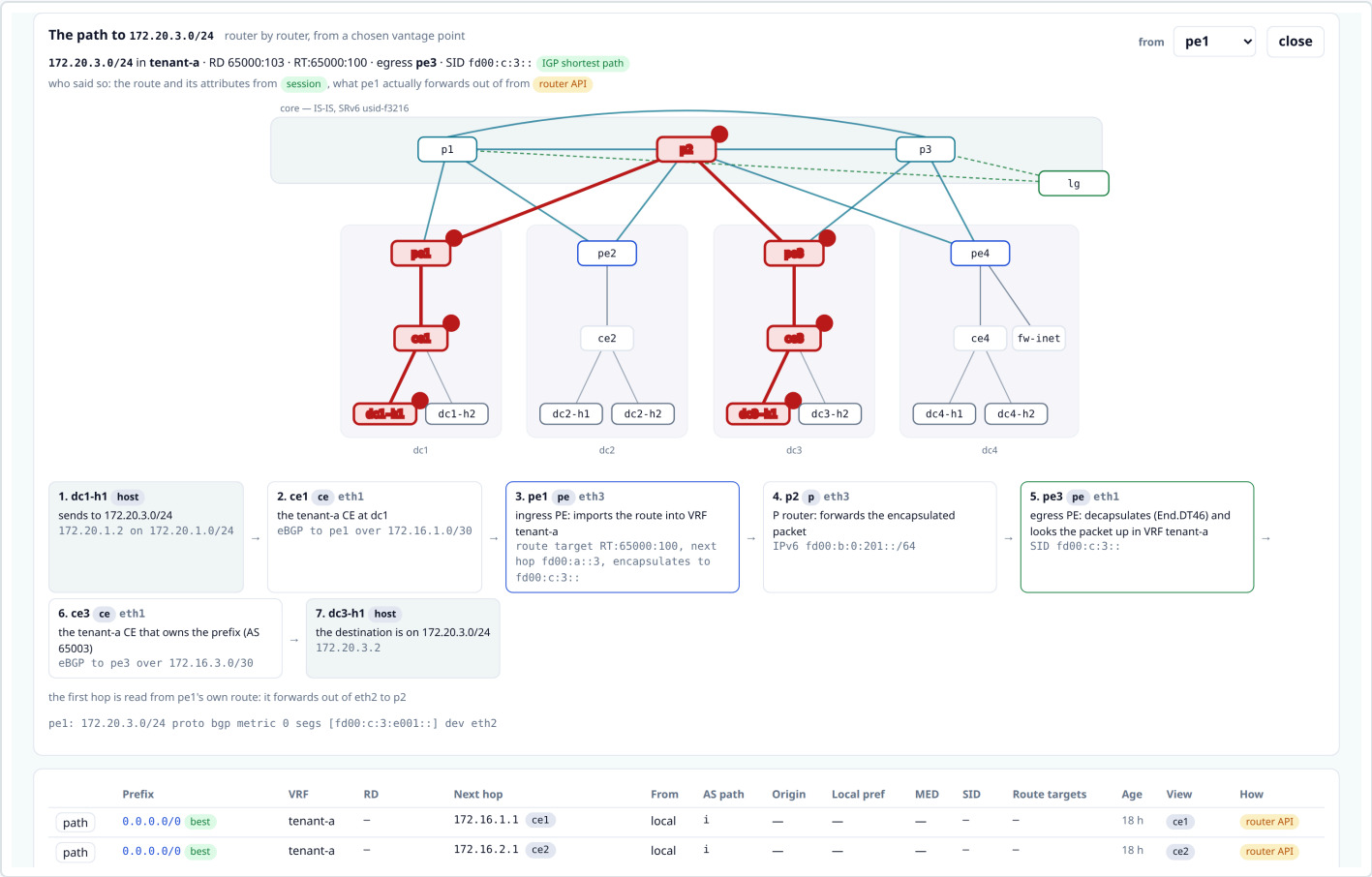
How it was read: router API — tenant-a as the routers themselves installed it

The core's table is one path per prefix per originating PE. A router's own tables are what survived import into the VRF and what selection actually installed in its RIB — different counts, legitimately, and the gap between them is where import and selection problems live.

A looking glass that silently merged the two, or showed one and called it the truth, would be one you could not trust. So the transport is a filter here, a column on every row, and a pill beside every view — everywhere in the product.

The path, router by router

For one prefix from a chosen vantage point: the map with the path lit up and its hops numbered, then a card per hop.



Two sources, and the card says which

The route and its attributes come from the session; what the router will really do with the packet comes from its own RIB.

Equal-cost paths are named rather than silently collapsed into one line.

A steered prefix is drawn along the segment list its policy installed — the uSID carrier is unpacked back into the routers it names.

Ask the same prefix from a different router and the diagram is rebuilt from that router's view.

dc1-h1 → 172.20.3.0/24: the CE's eBGP hand-off, pe1's import and SRv6 encapsulation, p2, pe3's End.DT46 decapsulation, ce3, the host

One prefix, every view at once

The core's VPN table, each reflector's and PE's own VPN table, each PE's and CE's VRF table, and every router's RIB — in one page.

the first hop is read from pe1's own route: it forwards out of eth2 to p2

pe1: 172.20.3.0/24 proto bgp metric 0 segs [fd00:c:3:e001::] dev eth2

The core's VPN table

as the reflectors handed it over

session

VRF	tenant-a	Route distinguisher	65000:103	Next hop	fd00:a::3	Advertised by	p1	Originating node	pe3	AS path	65003
Origin	IGP	Local pref	100	MED	0	Route targets	RT:65000:100	Originator-id	10.255.0.3	Cluster list	10.255.0.11
SRv6 SID	fd00:c:3::	Label	917520	Selection	best path (Neighbor IP)	First seen	9/24/2026, 1:53:15 PM	Last change	9/25/2026, 3:49:04 PM	State	in the table

VRF	tenant-a	Route distinguisher	65000:103	Next hop	fd00:a::3	Advertised by	p3	Originating node	pe3	AS path	65003
Origin	IGP	Local pref	100	MED	0	Route targets	RT:65000:100	Originator-id	10.255.0.3	Cluster list	10.255.0.13
SRv6 SID	fd00:c:3::	Label	917520	Selection	multipath	First seen	9/24/2026, 1:53:15 PM	Last change	9/25/2026, 3:49:04 PM	State	in the table

On pe1

after import, in the tenant's VRF and its routing table and its own VPN table

router SSHrouter API

VRF	tenant-a	Next hop	fd00:a::3	Advertised by	local	Originating node	pe3	AS path	65003	Origin	IGP
Local pref	100	MED	0	Route targets	RT:65000:100	Originator-id	10.255.0.3	Cluster list	10.255.0.11	SRv6 SID	fd00:c:3::
Transposed SID	fd00:c:3:e001::	Label	917520	Imported from	65000:103	Selection	best path (Neighbor IP)	First seen	9/24/2026, 1:53:16 PM	Last change	9/25/2026, 3:50:27 PM
State	in the table										

VRF	tenant-a	Next hop	fd00:a::3	Advertised by	local	Originating node	pe1	AS path	i (local)	SRv6 SID	segs
First seen	9/24/2026, 9:42:58 PM	Last change	9/25/2026, 3:56:24 PM	State	in the table	Weight	0	First seen	9/24/2026, 9:47:17 PM	Last change	9/25/2026, 3:50:26 PM

VRF	tenant-a	Route distinguisher	65000:103	Next hop	fd00:a::3	Advertised by	p1	Originating node	pe3	AS path	65003
Origin	IGP	Local pref	100	MED	0	Weight	0	Selection	best path (Neighbor IP)	First seen	9/24/2026, 9:47:17 PM
Last change	9/25/2026, 3:50:26 PM	State	in the table								

VRF	tenant-a	Next hop	fd00:a::3	Advertised by	local	Originating node	pe3	AS path	65003	Origin	IGP
Local pref	100	MED	0	Route targets	RT:65000:100	Originator-id	10.255.0.3	Cluster list	10.255.0.13	SRv6 SID	fd00:c:3::
Transposed SID	fd00:c:3:e001::	Label	917520	Imported from	65000:103	Selection	multipath	First seen	9/24/2026, 1:53:16 PM	Last change	9/25/2026, 3:50:27 PM
State	in the table										

VRF	tenant-a	Route distinguisher	65000:103	Next hop	fd00:a::3	Advertised by	p3	Originating node	pe3	AS path	65003
Origin	IGP	Local pref	100	MED	0	Weight	0	Selection	multipath	First seen	9/24/2026, 9:47:17 PM
Last change	9/25/2026, 3:50:26 PM	State	in the table								

Each with its full attribute set and a pill saying how it was read

Where the answers differ is the diagnosis

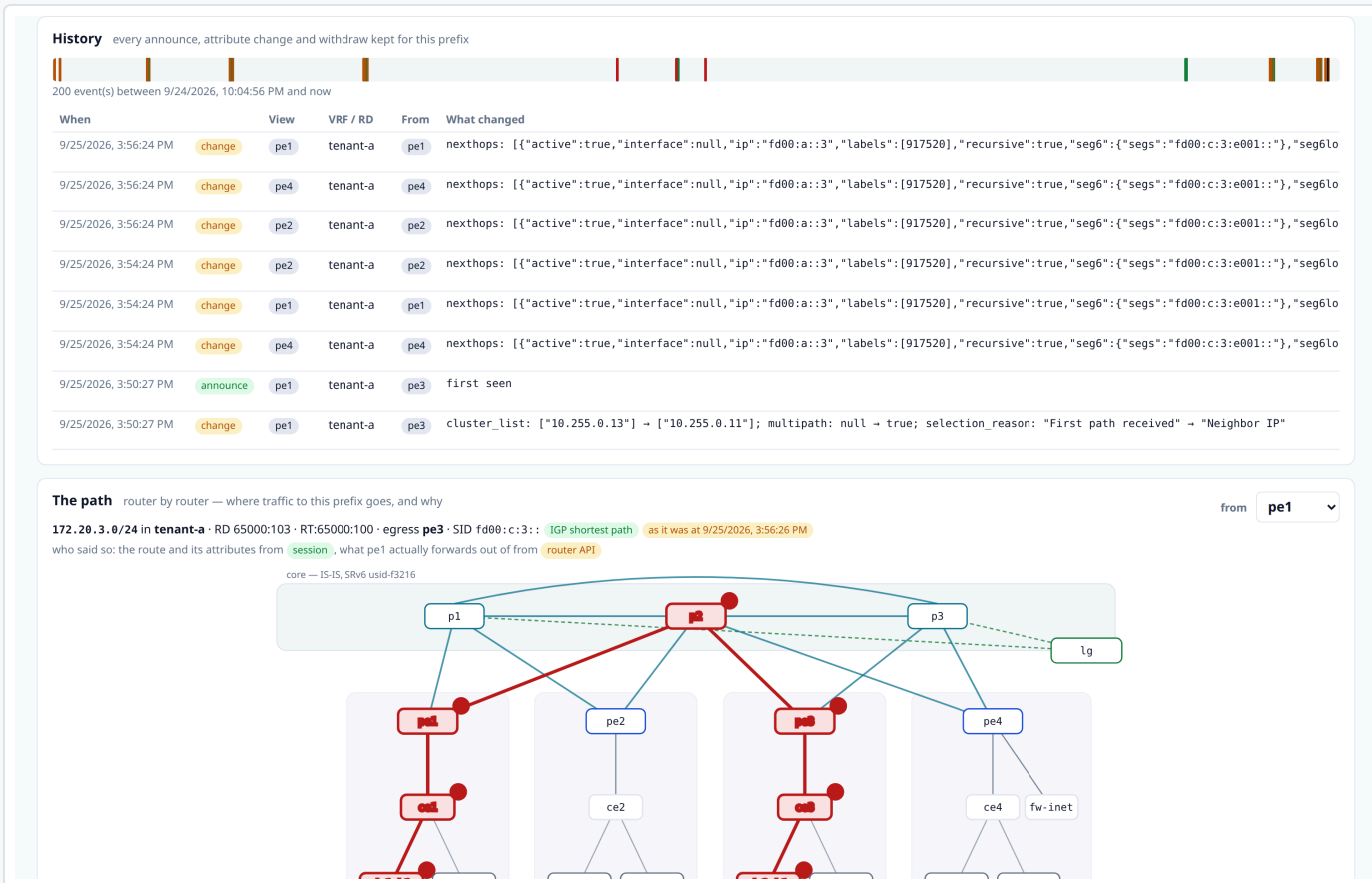
In the core but missing from a VRF: an import problem — a route target that does not match.

In the VRF but not in the RIB: a selection problem — something else won.

Attributes are shown whole: route targets, originator-id, cluster list, SID, transposed SID, label, and the selection reason the router gave.

Everything that has happened to it

Each collection is reconciled against what is stored, so the history is a log of events rather than a pile of snapshots.



168 events for this one prefix, each with the fields that changed

Reconciled, not sampled

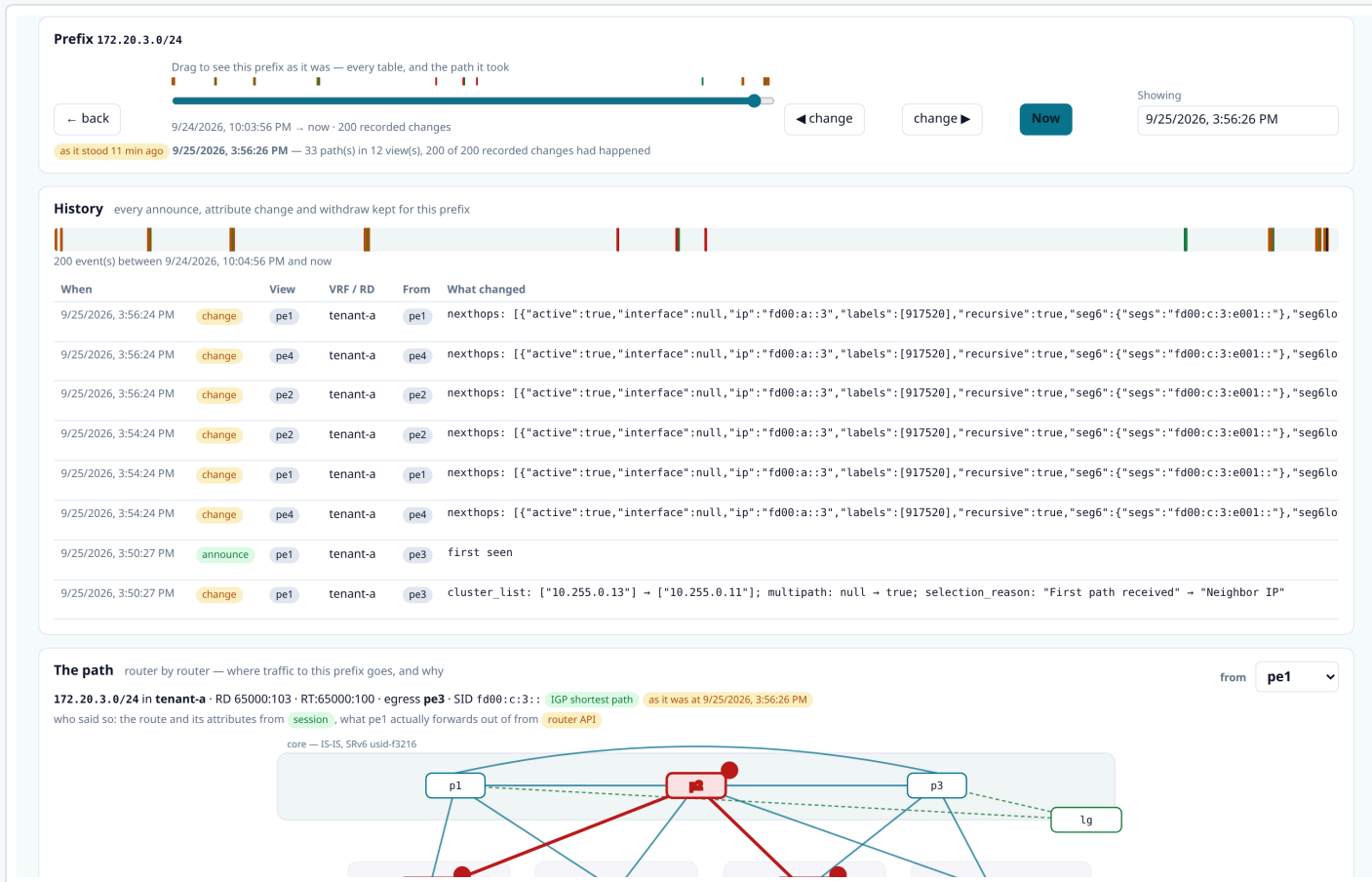
A path that is new raises an announce; one whose attributes differ raises a change carrying the difference — local_pref: 100 → 200; one that has gone raises a withdraw.

So the storage is proportional to what actually changed, not to how often it was polled.

Thirty days are kept and pruned hourly, and the database survives a restart of the collector or of the VM.

And the slider replays it

Drag it and every table on the page re-renders as it stood at that moment — and so does the path diagram.



Why the path matters here

The forwarding state comes out of the RIB history, not out of today's routers — so a policy that steered this prefix an hour ago is still drawn the way it steered it.

◀ and ▶ snap to the moment just after each recorded change, so nothing is missed between two drags of a slider.

GET /api/state?at=... and GET /api/path?...&at=... are the same replay, for anything that wants it.

“As it stood 31 minutes ago”: 33 paths in 12 views, with 168 of 168 recorded changes having happened

“When did this prefix go away?”

The change log for the whole core, newest first, filterable by prefix, by kind and by age.

BGP Looking Glass

a passive route collector in the srv6-core core (AS 65000, reflectors p1 + p3) — every VPN prefix, its attributes, and how they changed

Overview

Prefixes

History

Routers

Sessions

Live query

2/2 sessions · 76 prefixes · AS 65000 · [dark](#)

History

the whole table's changes, newest first

Prefix (optional)

Kind

Since

Show

172.20.3.0/24

withdraw

last day

When		Prefix	VRF	View	From	What changed
9/25/2026, 4:02:25 PM	withdraw	172.21.3.0/24	tenant-b	pe1	pe1	withdrawn
9/25/2026, 3:56:26 PM	withdraw	fd00:c:4::1/128	default	pe3	pe3	withdrawn
9/25/2026, 3:56:26 PM	withdraw	fd00:c:4::/48	default	pe3	pe3	withdrawn
9/25/2026, 3:56:26 PM	withdraw	fd00:c:2::1/128	default	pe3	pe3	withdrawn
9/25/2026, 3:56:26 PM	withdraw	fd00:c:2::/48	default	pe3	pe3	withdrawn
9/25/2026, 3:56:26 PM	withdraw	fd00:c:1::1/128	default	pe3	pe3	withdrawn
9/25/2026, 3:56:26 PM	withdraw	fd00:c:1::/48	default	pe3	pe3	withdrawn
9/25/2026, 3:56:26 PM	withdraw	fd00:c:12::1/128	default	pe3	pe3	withdrawn
9/25/2026, 3:56:26 PM	withdraw	fd00:c:12::/48	default	pe3	pe3	withdrawn
9/25/2026, 3:56:26 PM	withdraw	fd00:c:11::1/128	default	pe3	pe3	withdrawn
9/25/2026, 3:56:26 PM	withdraw	fd00:c:11::/48	default	pe3	pe3	withdrawn

The operational value

A prefix that is missing now tells you nothing about when it went, or which PE stopped originating it. This does.

The same log answers “what changed in the last hour?” across the whole core, which is the churn number on the front page.

Every event carries its view and its transport, so a withdraw seen only by one router is distinguishable from one the core agreed on.

Withdraws only — the question a looking glass without a memory cannot answer at all

And the classic button: a command on a router

Only show ..., ping and traceroute, and only on the lab's own devices — over the same HTTPS API the collector polls with.

Live query a `show` command straight on a router, or a ping from it

Device

pe1

Command

show bgp ipv4 vpn rd 65000:103 172.20.3.0/24

Run

Suggestions: [show bgp vrf tenant-a ipv4 unicast](#) · [show bgp vrf tenant-a ipv6 unicast](#) · [show bgp ipv4 vpn](#) · [show bgp summary](#) · [show bgp ipv4 vpn detail](#) · [show ip route vrf tenant-a](#) · [show isis neighbor](#) · [show bgp neighbors](#)

Only show ..., ping <address> [vrf X] [count N] and traceroute <address> are accepted, and only on the lab's own devices.

pe1: show bgp ipv4 vpn rd 65000:103 172.20.3.0/24 (0.72 s)

```
BGP routing table entry for 65000:103:172.20.3.0/24, version 119
not allocated
Paths: (2 available, best #1)
  Not advertised to any peer
  65003
    0.0.0.0 (metric 20) from fd00:a::11 (10.255.0.3)
      Origin IGP, metric 0, localpref 100, valid, internal, multipath, best (Neighbor IP)
      Extended Community: RT:65000:100
      Originator: 10.255.0.3, Cluster List: 10.255.0.11
      Remote labels: 917520
      Remote SID: fd00:c:3::, sid structure=[32 16 16 0 16 48]
      Last update: Fri Sep 25 19:48:49 2026
    65003
    0.0.0.0 (metric 20) from fd00:a::13 (10.255.0.3)
      Origin IGP, metric 0, localpref 100, valid, internal, multipath
      Extended Community: RT:65000:100
      Originator: 10.255.0.3, Cluster List: 10.255.0.13
      Remote labels: 917520
      Remote SID: fd00:c:3::, sid structure=[32 16 16 0 16 48]
      Last update: Fri Sep 25 17:56:56 2026
```

Narrow on purpose

No extra access path exists for the button: it is the router API already rendered into every configuration, with the same key and the same allow-client.

Three verbs, validated server-side, against the lab's own device list — a looking glass is a read-only instrument and this keeps it one.

And it is the tie-breaker: when the stored answer and the live box disagree, this says which one is stale.

pe1, asked for the route the collector's session carries — route target, originator, cluster list, remote SID and its structure

Everything the page does is the API

The page is one consumer of it; Prometheus is another; anything you write is a third.

```
$ curl -s http://10.3.0.70:8080/api/peers | jq '.peers[0]'
```

```
{
  "dropped": 0,
  "established_count": 1,
  "families": {
    "ipv4Vpn": {
      "accepted": 60,
      "advertised": null,
      "sent": 0
    },
    "ipv6Vpn": {
      "accepted": 16,
      "advertised": null,
      "sent": 0
    }
  },
  "graceful_restart": {
    "ipv4Vpn": true,
    "ipv6Vpn": true
  },
  "hostname": "p1",
  "ip": "fd00:b:0:121::1",
  "local": "fd00:b:0:121::2",
  "messages": {
    "rx": 1240,
    "tx": 831
  },
  ...
}
```

GET /api/peers, as the page itself asks for it

The read API

/api/prefixes and /api/prefix — the table, and everything known about one prefix in every view.

/api/path — the hop-by-hop reconstruction, from a vantage point you name.

/api/state?at=... and /api/path?...&at=... — the same answers, replayed to any moment in the window.

/api/history, /api/routers, /api/peers and /api/query — and no write verb anywhere in it.

...and the numbers go to Prometheus

Paths and prefixes per node, source, transport, family, VRF and table — plus churn, session state and collection health.

```
# HELP lg_up the looking glass is running
# TYPE lg_up gauge
lg_up{lab="srv6-core",node="lg"} 1
lg_paths{lab="srv6-core",node="lg",source="cel",via="router-api",afi="ipv4",safi="rib",vrf="default"} 3
lg_prefixes{lab="srv6-core",node="lg",source="cel",via="router-api",afi="ipv4",safi="rib",vrf="default"} 3
lg_paths{lab="srv6-core",node="lg",source="cel",via="router-api",afi="ipv4",safi="rib",vrf="tenant-a"} 63
lg_prefixes{lab="srv6-core",node="lg",source="cel",via="router-api",afi="ipv4",safi="rib",vrf="tenant-a"} 52
lg_paths{lab="srv6-core",node="lg",source="cel",via="router-api",afi="ipv4",safi="rib",vrf="tenant-b"} 13
lg_prefixes{lab="srv6-core",node="lg",source="cel",via="router-api",afi="ipv4",safi="rib",vrf="tenant-b"} 12
lg_paths{lab="srv6-core",node="lg",source="cel",via="router-api",afi="ipv6",safi="rib",vrf="default"} 6
lg_prefixes{lab="srv6-core",node="lg",source="cel",via="router-api",afi="ipv6",safi="rib",vrf="default"} 1
lg_paths{lab="srv6-core",node="lg",source="cel",via="router-api",afi="ipv6",safi="rib",vrf="tenant-a"} 15
lg_prefixes{lab="srv6-core",node="lg",source="cel",via="router-api",afi="ipv6",safi="rib",vrf="tenant-a"} 12
lg_paths{lab="srv6-core",node="lg",source="cel",via="router-api",afi="ipv6",safi="rib",vrf="tenant-b"} 14
lg_prefixes{lab="srv6-core",node="lg",source="cel",via="router-api",afi="ipv6",safi="rib",vrf="tenant-b"} 12
lg_paths{lab="srv6-core",node="lg",source="cel",via="router-ssh",afi="ipv4",safi="unicast",vrf="tenant-a"} 50
lg_prefixes{lab="srv6-core",node="lg",source="cel",via="router-ssh",afi="ipv4",safi="unicast",vrf="tenant-a"} 50
lg_paths{lab="srv6-core",node="lg",source="cel",via="router-ssh",afi="ipv4",safi="unicast",vrf="tenant-b"} 10
lg_prefixes{lab="srv6-core",node="lg",source="cel",via="router-ssh",afi="ipv4",safi="unicast",vrf="tenant-b"} 10
lg_paths{lab="srv6-core",node="lg",source="cel",via="router-ssh",afi="ipv6",safi="unicast",vrf="tenant-a"} 8
lg_prefixes{lab="srv6-core",node="lg",source="cel",via="router-ssh",afi="ipv6",safi="unicast",vrf="tenant-a"} 8
```

GET /metrics, scraped by the shared lab monitoring on the NMS

One row per view

The same breakdown the page shows, as a time series — so “when did pe3 stop holding tenant-a?” has a graph as well as a change log.

A dashboard row

The shared Grafana has a looking-glass row built from these: paths over time, per transport, and the age of each router's last collection.

And three alerts

A reflector session down, a collection failing, and churn above its threshold — beside every other alert for the host.

What it is for

Three properties, and every one of them is a choice that shows up in the product.

The attributes, unaltered

A reflector client sees the update the PE originated: route distinguisher, route targets, SRv6 SID, label, originator-id, cluster list, AS path. Nothing is parsed out of a screen, and extended-nexthop keeps the originating PE as the next hop.

Every answer, labelled

The core's table, each router's own tables and each router's RIB, side by side — with the transport each came over, because the same prefix seen two ways is two answers and hiding which is which makes both useless.

And yesterday's answer too

Reconciled history rather than samples: thirty days of announces, changes and withdraws, with every table and the path itself re-drawn for any moment in that window.

<http://192.168.50.231:8092> · the lab-side address is <http://10.3.0.70:8080> · `./lab.sh lg status` · the code is `lg/` in this repository.
A 3½-minute walkthrough of the same ground is [docs/demo/lg-demo.mp4](#); the portal and the core are in [docs/srv6-workflows.pptx](#).